PHASE 0 — Repository Inspection and Project Foundation

You are working on the Agentic Pharmacology Research Assistant described in the project context.

Your task is to prepare the existing repository for implementation.

Objective

Inspect the current project and establish a clean, secure, reproducible development foundation.

Do not implement application features yet.

Step 1 — Inspect the repository

Inspect:

- Existing directory structure.
- Existing Python files.
- Existing Dockerfile.
- `.gitignore`.
- `.env.example`.
- `.env` without printing or exposing secret values.
- README.
- Git branch, remote, and working-tree status.
- Python version and virtual environment.
- Available Node.js and npm versions.

Do not assume that empty directories or existing files are correctly configured.

Step 2 — Review project configuration

Verify:

- `.env` is excluded from Git.
- `.env.example` contains the required variable names.
- No actual API credentials are committed.
- Python virtual environment is correctly configured.
- The repository structure supports the planned architecture.

Required environment variable names:

- OPENAI_API_KEY
- OPENAI_BASE_URL
- OPENAI_MODEL
- OPENAI_EMBEDDING_MODEL

Do not invent actual model names or credentials.

Step 3 — Dependency strategy

Propose a minimal initial dependency list.

Consider:

- openai
- python-dotenv
- numpy
- fastapi
- uvicorn
- pydantic
- pytest
- httpx

Do not install every dependency automatically without explaining the purpose and compatibility.

The project uses Python 3.13.5 currently. Check compatibility.

Step 4 — Corpus inspection

If the corpus files are available locally, inspect:

- Number of documents.
- File extensions.
- Encoding.
- Document sizes.
- Heading structures.
- Whether documents contain repeated or unusual formatting.

Do not modify the original corpus.

If the corpus is not yet available, document this as an outstanding input. Do not invent its structure.

Step 5 — Documentation

Create or update a concise README containing:

- Project overview.
- Technology stack.
- Prerequisites.
- Initial setup.
- Environment configuration.
- Current implementation status.

Do not document functionality that has not been implemented.

Step 6 — Testing and validation

Run appropriate environment and configuration checks.

Verify the project can use its virtual environment and Python package manager.

Constraints

- Do not implement ingestion, embeddings, retrieval, RAG, or agent logic.
- Do not implement the frontend.
- Do not introduce LangChain or LangGraph.
- Do not replace the existing architecture without justification.
- Do not commit automatically.
- Do not expose secrets.

Deliverables

At the end, provide:

1. Repository inspection summary.
2. Problems found.
3. Files created or modified.
4. Dependency recommendations.
5. Corpus inspection findings or missing-input notice.
6. Commands used for verification.
7. Test results.
8. A clear list of remaining tasks.